

Materiały źródłowe

- Dane do analizy pochodzą ze zbioru "Cardiotocography Data Set" dostępnego w UCI Machine Learning Repository: D. Campos and J. Bernardes. "Cardiotocography," UCI Machine Learning Repository, 2000. [Online]. URL: <https://doi.org/10.24432/C51S4N>
- Dokumentacja Scikit-learn: <https://scikit-learn.org/stable/documentation.html>

Wykorzystane biblioteki

Podczas realizacji zadania wykorzystano następujące biblioteki języka Python:

- **Pandas:** Do wczytywania, manipulowania i analizy danych tabelarycznych (DataFrame).
- **Scikit-learn (sklearn):** Kluczowa biblioteka do uczenia maszynowego, wykorzystana do:
 - **SimpleImputer:** Uzupełniania brakujących wartości.
 - **train_test_split:** Podziału danych na zbiory treningowe i testowe.
 - **StandardScaler, Normalizer:** Standaryzacji i normalizacji cech.
 - **PCA:** Analizy głównych składowych do redukcji wymiarowości.
 - **DecisionTreeClassifier, GaussianNB, RandomForestClassifier, SVC:** Implementacje algorytmów klasyfikacyjnych.
 - **accuracy_score, precision_score, recall_score, confusion_matrix, classification_report:** Metryki oceny jakości klasyfikatorów.

Wprowadzenie do metryk oceny klasyfikacji

Wyróżniamy cztery możliwe wyniki dla pojedynczej próbki:

- **TP (True Positive):** Próbka pozytywna poprawnie zaklasyfikowana jako pozytywna.
- **FP (False Positive):** Próbka negatywna błędnie zaklasyfikowana jako pozytywna (błąd typu I).
- **FN (False Negative):** Próbka pozytywna błędnie zaklasyfikowana jako negatywna (błąd typu II).
- **TN (True Negative):** Próbka negatywna poprawnie zaklasyfikowana jako negatywna.

Na podstawie tych wartości budowana jest **Macierz Pomyłek (Confusion Matrix)**.

Kluczowe metryki:

- **Accuracy (Dokładność):**
 - **Definicja:** Stosunek liczby poprawnie zaklasyfikowanych próbek (TP + TN) do całkowitej liczby próbek.

- **Wzór (binarny):** $(TP + TN) / (TP + TN + FP + FN)$
- **Interpretacja:** Ogólna miara skuteczności modelu. Może być myląca w przypadku niebalansowanych zbiorów danych, gdzie model faworyzujący klasę większościową może osiągnąć wysoką dokładność, mimo słabej predykcji klas mniejszościowych.
- **Precision (Precyzja):**
 - **Definicja:** Stosunek liczby prawdziwie pozytywnych predykcji (TP) do całkowitej liczby predykcji pozytywnych (TP + FP).
 - **Wzór (binarny):** $TP / (TP + FP)$
 - **Interpretacja:** Z wszystkich próbek, które model oznaczył jako pozytywne, jaki procent faktycznie był pozytywny? Wysoka precyzja oznacza, że model rzadko myli się, gdy przewiduje klasę pozytywną.
- **Recall (Czułość, Pełność, True Positive Rate - TPR):**
 - **Definicja:** Stosunek liczby prawdziwie pozytywnych predykcji (TP) do całkowitej liczby faktycznie pozytywnych próbek (TP + FN).
 - **Wzór (binarny):** $TP / (TP + FN)$
 - **Interpretacja:** Z wszystkich faktycznie pozytywnych próbek, jaki procent model był w stanie poprawnie zidentyfikować? Wysoka czułość oznacza, że model dobrze wykrywa próbki należące do klasy pozytywnej.
- **F1-score (Miara F1):**
 - **Definicja:** Średnia harmoniczna precyzji i czułości.
 - **Wzór (binarny):** $2 * (Precision * Recall) / (Precision + Recall)$
 - **Interpretacja:** Stanowi kompromis między precyzją a czułością. Jest szczególnie użyteczna, gdy obie te metryki są ważne, a także w przypadku niebalansowanych klas, gdzie prosta dokładność może być niewystarczająca. Wartość F1-score mieści się w zakresie [0, 1], gdzie 1 oznacza idealną precyzję i czułość.

Metryki w kontekście wieloklasowym:

W problemach wieloklasowych, takich jak analizowany w tym raporcie (10 klas wzorców morfologicznych płodu), powyższe metryki są zazwyczaj obliczane dla każdej klasy z osobna (traktując daną klasę jako "pozytywną", a wszystkie pozostałe jako "negatywne" - podejście "one-vs-rest"). Następnie można je agregować, aby uzyskać ogólny obraz działania modelu:

- **Macro Average:** Oblicza metrykę niezależnie dla każdej klasy, a następnie uśrednia wyniki, nie uwzględniając licznosci klas. Każda klasa ma taką samą wagę.
- **Weighted Average:** Oblicza metrykę dla każdej klasy, a następnie uśrednia wyniki, ważone liczbą prawdziwych instancji dla każdej klasy. Daje to obraz działania modelu z uwzględnieniem niebalansowania klas.
- **Micro Average:** Agreguje wkłady wszystkich klas, aby obliczyć średnią metrykę. W przypadku dokładności, precyzji, czułości i F1-score, micro-average będzie miał taką samą wartość.

W tym raporcie będziemy analizować zarówno metryki dla poszczególnych klas, jak i ich agregacje (macro avg, weighted avg), aby uzyskać pełny obraz skuteczności testowanych klasyfikatorów.

Domena problemu

Problem dotyczy identyfikacji typu wzoru morfologicznego płodu na podstawie cech diagnostycznych kardiogramu (CTG, z ang. cardiotocogram). Źródło danych zawiera 2126 wierszy danych kardiogramów płodu (CTG). Celem jest przeprowadzenie klasyfikacji 10 typów wzorców morfologicznych płodu na podstawie 21 cech diagnostycznych. W cechach występują pewne brakujące wartości. Kolumny w pliku to:

- LB - [cecha] wartość bazowa FHR (uderzeń na minutę)
- AC - [cecha] liczba przyspieszeń na sekundę
- FM - [cecha] liczba ruchów płodu na sekundę
- UC - [cecha] liczba skurczów macicy na sekundę
- DL - [cecha] liczba łagodnych spowolnień na sekundę
- DS - [cecha] liczba poważnych spowolnień na sekundę
- DP - [cecha] liczba długotrwałych spowolnień na sekundę
- ASTV - [cecha] odsetek czasu z nieprawidłową zmiennością krótkoterminową
- MSTV - [cecha] średnia wartość zmienności krótkoterminowej
- ALTV - [cecha] odsetek czasu z nieprawidłową zmiennością długoterminową
- MLTV - [cecha] średnia wartość zmienności długoterminowej
- Width - [cecha] szerokość histogramu FHR
- Min - [cecha] minimum histogramu FHR
- Max - [cecha] maksimum histogramu FHR
- Nmax - [cecha] liczba pików histogramu
- Nzeros - [cecha] liczba zer histogramu
- Mode - [cecha] modus histogramu
- Mean - [cecha] średnia histogramu
- Median - [cecha] mediana histogramu
- Variance - [cecha] wariancja histogramu
- Tendency - [cecha] tendencja histogramu
- CLASS - [cel] kod klasy wzorca morfologicznego płodu (od 1 do 10)

Ładowanie danych

```
In [69]: from sklearn.metrics import accuracy_score, precision_score, recall_score, confu
from sklearn.preprocessing import Normalizer, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.impute import SimpleImputer
from sklearn.decomposition import PCA
from sklearn.svm import SVC
```

```

from pandas import DataFrame
import pandas as pd

pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_columns', None)

seed = 68

columns_to_load: list[str] = [
    "LB", "AC.1", "FM.1", "UC.1", "DL.1", "DS.1", "DP.1", "ASTV",
    "MSTV", "ALTV", "MLTV", "Width", "Min", "Max", "Nmax", "Nzeros",
    "Mode", "Mean", "Median", "Variance", "Tendency", "CLASS"
]

df: pd.DataFrame = pd.read_excel(
    "CTG.xls", sheet_name="Data", skiprows=1, usecols=columns_to_load)

df.rename(columns={col: col.replace('.1', '')
                  for col in df.columns if '.1' in col}, inplace=True)

```

Eksploracja danych

```

In [70]: stats: DataFrame = df.describe().T
missing = df.isnull().sum()
stats['missing'] = missing
stats

```

Out[70]:

	count	mean	std	min	25%	50%	75%	
LB	2126.0	133.303857	9.840844	106.0	126.000000	133.000000	140.000000	160
AC	2126.0	0.003170	0.003860	0.0	0.000000	0.001630	0.005631	0
FM	2127.0	0.009695	0.047764	0.0	0.000000	0.000000	0.002527	0
UC	2127.0	0.004362	0.002949	0.0	0.001877	0.004484	0.006527	0
DL	2128.0	0.001890	0.002976	0.0	0.000000	0.000000	0.003270	0
DS	2128.0	0.000004	0.000069	0.0	0.000000	0.000000	0.000000	0
DP	2128.0	0.000159	0.000590	0.0	0.000000	0.000000	0.000000	0
ASTV	2127.0	47.008933	17.210648	12.0	32.000000	49.000000	61.000000	85
MSTV	2127.0	1.335449	0.891543	0.2	0.700000	1.200000	1.700000	7
ALTV	2127.0	9.884814	18.476534	0.0	0.000000	0.000000	11.000000	95
MLTV	2127.0	8.207616	5.701926	0.0	4.600000	7.400000	10.800000	50
Width	2126.0	70.445908	38.955693	3.0	37.000000	67.500000	100.000000	180
Min	2126.0	93.579492	29.560212	50.0	67.000000	93.000000	120.000000	150
Max	2126.0	164.025400	17.944183	122.0	152.000000	162.000000	174.000000	230
Nmax	2126.0	4.068203	2.949386	0.0	2.000000	3.000000	6.000000	10
Nzeros	2126.0	0.323612	0.706059	0.0	0.000000	0.000000	0.000000	10
Mode	2126.0	137.452023	16.381289	60.0	129.000000	139.000000	148.000000	180
Mean	2126.0	134.610536	15.593596	73.0	125.000000	136.000000	145.000000	180
Median	2126.0	138.090310	14.466589	77.0	129.000000	139.000000	148.000000	180
Variance	2126.0	18.808090	28.977636	0.0	2.000000	7.000000	24.000000	260
Tendency	2126.0	0.320320	0.610829	-1.0	0.000000	0.000000	1.000000	1
CLASS	2126.0	4.509878	3.026883	1.0	2.000000	4.000000	7.000000	10



- Zbiór danych zawiera 2126 obserwacji i 21 cech diagnostycznych oraz kolumnę z klasą (CLASS).
- Brakujące dane: W większości cech występują pojedyncze brakujące wartości (1–3 na cechę), co stanowi bardzo mały odsetek i można je łatwo uzupełnić lub usunąć.
- Statystyki cech: Większość cech ma wartości bliskie zeru lub niskie średnie, co sugeruje, że dane są zróżnicowane i mogą wymagać standaryzacji.
- Kolumna CLASS: Obejmuje 10 klas (od 1 do 10), średnia to 4,5, a rozkład klas należy sprawdzić pod kątem zbalansowania.

Wnioski: Dane są kompletne, dobrze opisane i gotowe do dalszej analizy po uzupełnieniu braków. Warto rozważyć normalizację cech przed budową modeli klasyfikacyjnych.

```
In [71]: class_counts = df['CLASS'].value_counts().sort_index()
print("Liczność poszczególnych klas:\n", class_counts)
```

Liczność poszczególnych klas:

```
CLASS
1.0    384
2.0    579
3.0     53
4.0     81
5.0     72
6.0    332
7.0    252
8.0    107
9.0     69
10.0   197
```

Name: count, dtype: int64

Dalsze uwagi dotyczące danych:

- **Typy danych:** Wszystkie analizowane cechy są numeryczne (typu float lub int), co upraszcza przygotowanie danych do większości modeli uczenia maszynowego.
- **Rozkład klas (CLASS):** Jak widać w licznosciach, klasy są niezbilansowane. Klasy 2 (579 obserwacji), 1 (384) i 6 (332) są najliczniejsze, podczas gdy klasy takie jak 3 (53), 9 (69) czy 5 (72) mają znacznie mniej reprezentantów. Niezbilansowanie klas może prowadzić do sytuacji, w której modele będą faworyzować klasy większościowe, osiągając wysoką dokładność ogólną, ale słabo radząc sobie z predykcją klas mniejszościowych. Może to wymagać zastosowania technik radzenia sobie z niezbilansowanymi danymi (np. oversampling, undersampling, ważenie klas) w bardziej zaawansowanych analizach.
- **Zakresy wartości cech:** Różne cechy mają różne zakresy wartości i skale (np. LB w zakresie 106-160, podczas gdy AC ma wartości bliskie zeru). To sugeruje, że standaryzacja lub normalizacja mogą być korzystne dla niektórych algorytmów (np. SVM, PCA, algorytmy oparte na odległości).

Przygotowanie danych

```
In [72]: df = df.dropna(subset=['CLASS'])

X = df.drop(columns=['CLASS'])
y = df['CLASS']

imputer = SimpleImputer(strategy='median')
X_imputed = imputer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size=0.2,
```

```
In [73]: # klasyfikacja bez przetwarzania
clf = DecisionTreeClassifier(random_state=seed)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
acc_raw = accuracy_score(y_test, y_pred)
print("Dokładność bez przetwarzania:", acc_raw)
```

```

# standaryzacja
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)
clf_std = DecisionTreeClassifier(random_state=seed)
clf_std.fit(X_train_std, y_train)
y_pred_std = clf_std.predict(X_test_std)
acc_std = accuracy_score(y_test, y_pred_std)
print("Dokładność po standaryzacji:", acc_std)

# normalizacja
normalizer = Normalizer()
X_train_norm = normalizer.fit_transform(X_train)
X_test_norm = normalizer.transform(X_test)
clf_norm = DecisionTreeClassifier(random_state=seed)
clf_norm.fit(X_train_norm, y_train)
y_pred_norm = clf_norm.predict(X_test_norm)
acc_norm = accuracy_score(y_test, y_pred_norm)
print("Dokładność po normalizacji:", acc_norm)

# PCA (czyli redukcja do np. 10 komponentów)
pca = PCA(n_components=10, random_state=seed)
X_train_pca = pca.fit_transform(X_train_std)
X_test_pca = pca.transform(X_test_std)
clf_pca = DecisionTreeClassifier(random_state=seed)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
acc_pca = accuracy_score(y_test, y_pred_pca)
print("Dokładność po PCA:", acc_pca)

```

Dokładność bez przetwarzania: 0.8356807511737089

Dokładność po standaryzacji: 0.8356807511737089

Dokładność po normalizacji: 0.7652582159624414

Dokładność po PCA: 0.6737089201877934

Uzasadnienie kroków przygotowania danych:

- **Usuwanie wierszy z brakującą klasą:** Wiersze, dla których brakowało etykiety `CLASS`, zostały usunięte, ponieważ nie można ich wykorzystać ani do treningu, ani do wiarygodnej oceny modelu.
- **Imputacja brakujących wartości:** Do uzupełnienia brakujących wartości w cechach (`X`) użyto strategii `median`. Mediana jest mniej wrażliwa na wartości odstające niż średnia, co czyni ją wytrzymalszym wyborem dla danych, których rozkład nie jest idealnie normalny lub które mogą zawierać anomalie.
- **Podział na zbiór treningowy i testowy:** Dane podzielono w proporcji 80:20 (`test_size=0.2`). Użyto `stratify=y`, aby zapewnić, że proporcje klas w zbiorze treningowym i testowym będą zbliżone do proporcji w oryginalnym zbiorze danych. Jest to szczególnie ważne przy niezbalansowanych klasach. `random_state` zapewnia powtarzalność podziału.
- **Standaryzacja:** Przeskalowuje dane tak, aby miały średnią 0 i odchylenie standardowe 1. Jest często wymagana lub korzystna dla algorytmów wrażliwych na skalę cech (np. SVM, PCA, niektóre sieci neuronowe).
- **Normalizacja:** Przeskalowuje wektory cech dla każdej próbki (wiersza) tak, aby miały normę jednostkową (np. L2). Może być przydatna, gdy kierunek wektora cech jest

ważniejszy niż jego długość.

- **PCA (Primary Component Analysis):** Zastosowano PCA w celu redukcji wymiarowości do 10 komponentów. PCA identyfikuje kierunki (główne składowe) w danych, które wyjaśniają największą wariancję. Redukcja wymiarowości może pomóc w zwalczaniu "klątwy wymiarowości", przyspieszyć trening modeli i czasami poprawić generalizację poprzez usunięcie szumu. Wybór 10 komponentów jest arbitralny w tym miejscu; w praktyce często analizuje się skumulowaną wyjaśnioną wariancję, aby wybrać odpowiednią liczbę komponentów. Tutaj PCA zastosowano na danych po standaryzacji, co jest zalecaną praktyką.

Klasyfikatory

```
In [74]: datasets = {
    "raw": (X_train, X_test),
    "standardized": (X_train_std, X_test_std),
    "normalized": (X_train_norm, X_test_norm),
    "pca": (X_train_pca, X_test_pca)
}

decision_tree_hyper_params = [
    {"max_depth": None, "min_samples_split": 2},
    {"max_depth": 5, "min_samples_split": 5},
    {"max_depth": 10, "min_samples_split": 10}
]

naive_byes_hyper_params: list[dict[str, float]] = [
    {"var_smoothing": 1e-9},
    {"var_smoothing": 1e-8},
    {"var_smoothing": 1e-7}
]

results = []

for name, (Xtr, Xte) in datasets.items():
    for params in decision_tree_hyper_params:
        clf = DecisionTreeClassifier(random_state=seed, **params)
        clf.fit(Xtr, y_train)
        y_pred = clf.predict(Xte)
        acc = accuracy_score(y_test, y_pred)
        results.append(
            ("DecisionTree", name, str(params), acc)
        )
    print(f"\nDecisionTree ({name}, {params})")
    print(classification_report(y_test, y_pred, zero_division=0))
```


DecisionTree (raw, {'max_depth': None, 'min_samples_split': 2})

	precision	recall	f1-score	support
1.0	0.83	0.77	0.80	77
2.0	0.88	0.91	0.89	116
3.0	0.60	0.82	0.69	11
4.0	0.85	0.69	0.76	16
5.0	0.44	0.50	0.47	14
6.0	0.88	0.88	0.88	67
7.0	0.83	0.84	0.83	51
8.0	0.86	0.86	0.86	21
9.0	0.93	0.93	0.93	14
10.0	0.84	0.82	0.83	39
accuracy			0.84	426
macro avg	0.79	0.80	0.79	426
weighted avg	0.84	0.84	0.84	426

DecisionTree (raw, {'max_depth': 5, 'min_samples_split': 5})

	precision	recall	f1-score	support
1.0	0.71	0.77	0.74	77
2.0	0.83	0.94	0.88	116
3.0	0.62	0.91	0.74	11
4.0	0.67	0.12	0.21	16
5.0	0.45	0.36	0.40	14
6.0	0.95	0.81	0.87	67
7.0	0.81	0.92	0.86	51
8.0	1.00	0.76	0.86	21
9.0	1.00	0.57	0.73	14
10.0	0.76	0.82	0.79	39
accuracy			0.80	426
macro avg	0.78	0.70	0.71	426
weighted avg	0.81	0.80	0.79	426

DecisionTree (raw, {'max_depth': 10, 'min_samples_split': 10})

	precision	recall	f1-score	support
1.0	0.84	0.83	0.84	77
2.0	0.86	0.93	0.90	116
3.0	0.67	0.91	0.77	11
4.0	0.91	0.62	0.74	16
5.0	0.55	0.43	0.48	14
6.0	0.89	0.88	0.89	67
7.0	0.80	0.84	0.82	51
8.0	1.00	0.86	0.92	21
9.0	1.00	0.93	0.96	14
10.0	0.86	0.82	0.84	39
accuracy			0.85	426
macro avg	0.84	0.81	0.82	426
weighted avg	0.85	0.85	0.85	426

DecisionTree (standardized, {'max_depth': None, 'min_samples_split': 2})

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1.0	0.83	0.77	0.80	77
2.0	0.88	0.91	0.89	116
3.0	0.60	0.82	0.69	11
4.0	0.85	0.69	0.76	16
5.0	0.44	0.50	0.47	14
6.0	0.88	0.88	0.88	67
7.0	0.83	0.84	0.83	51
8.0	0.86	0.86	0.86	21
9.0	0.93	0.93	0.93	14
10.0	0.84	0.82	0.83	39
accuracy			0.84	426
macro avg	0.79	0.80	0.79	426
weighted avg	0.84	0.84	0.84	426

DecisionTree (standardized, {'max_depth': 5, 'min_samples_split': 5})

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1.0	0.71	0.77	0.74	77
2.0	0.83	0.94	0.88	116
3.0	0.62	0.91	0.74	11
4.0	0.67	0.12	0.21	16
5.0	0.45	0.36	0.40	14
6.0	0.95	0.81	0.87	67
7.0	0.81	0.92	0.86	51
8.0	1.00	0.76	0.86	21
9.0	1.00	0.57	0.73	14
10.0	0.76	0.82	0.79	39
accuracy			0.80	426
macro avg	0.78	0.70	0.71	426
weighted avg	0.81	0.80	0.79	426

DecisionTree (standardized, {'max_depth': 10, 'min_samples_split': 10})

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1.0	0.84	0.83	0.84	77
2.0	0.86	0.93	0.90	116
3.0	0.67	0.91	0.77	11
4.0	0.91	0.62	0.74	16
5.0	0.55	0.43	0.48	14
6.0	0.89	0.88	0.89	67
7.0	0.80	0.84	0.82	51
8.0	1.00	0.86	0.92	21
9.0	1.00	0.93	0.96	14
10.0	0.86	0.82	0.84	39
accuracy			0.85	426
macro avg	0.84	0.81	0.82	426
weighted avg	0.85	0.85	0.85	426

DecisionTree (normalized, {'max_depth': None, 'min_samples_split': 2})

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1.0	0.66	0.75	0.70	77
2.0	0.85	0.84	0.85	116
3.0	0.62	0.73	0.67	11

4.0	0.73	0.69	0.71	16
5.0	0.43	0.43	0.43	14
6.0	0.86	0.85	0.86	67
7.0	0.80	0.80	0.80	51
8.0	0.75	0.71	0.73	21
9.0	0.69	0.64	0.67	14
10.0	0.74	0.59	0.66	39
accuracy			0.77	426
macro avg	0.71	0.70	0.71	426
weighted avg	0.77	0.77	0.77	426

DecisionTree (normalized, {'max_depth': 5, 'min_samples_split': 5})

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1.0	0.62	0.75	0.68	77
2.0	0.88	0.91	0.89	116
3.0	0.57	0.36	0.44	11
4.0	0.90	0.56	0.69	16
5.0	0.23	0.21	0.22	14
6.0	0.86	0.91	0.88	67
7.0	0.81	0.76	0.79	51
8.0	0.71	0.57	0.63	21
9.0	0.88	0.50	0.64	14
10.0	0.63	0.62	0.62	39
accuracy			0.76	426
macro avg	0.71	0.62	0.65	426
weighted avg	0.76	0.76	0.75	426

DecisionTree (normalized, {'max_depth': 10, 'min_samples_split': 10})

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1.0	0.68	0.78	0.73	77
2.0	0.87	0.91	0.89	116
3.0	0.64	0.64	0.64	11
4.0	0.71	0.62	0.67	16
5.0	0.50	0.29	0.36	14
6.0	0.90	0.85	0.88	67
7.0	0.79	0.86	0.82	51
8.0	0.88	0.67	0.76	21
9.0	0.64	0.64	0.64	14
10.0	0.74	0.67	0.70	39
accuracy			0.79	426
macro avg	0.74	0.69	0.71	426
weighted avg	0.79	0.79	0.79	426

DecisionTree (pca, {'max_depth': None, 'min_samples_split': 2})

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

1.0	0.57	0.62	0.60	77
2.0	0.74	0.78	0.76	116
3.0	0.56	0.45	0.50	11
4.0	0.60	0.38	0.46	16
5.0	0.20	0.21	0.21	14
6.0	0.73	0.76	0.74	67

7.0	0.71	0.67	0.69	51			
8.0	0.89	0.81	0.85	21			
9.0	0.86	0.86	0.86	14			
10.0	0.58	0.54	0.56	39			
accuracy				0.67	426		
macro avg				0.64	0.61	0.62	426
weighted avg				0.67	0.67	0.67	426

```
DecisionTree (pca, {'max_depth': 5, 'min_samples_split': 5})
precision    recall  f1-score   support
```

1.0	0.57	0.68	0.62	77
2.0	0.62	0.83	0.71	116
3.0	0.33	0.09	0.14	11
4.0	0.50	0.06	0.11	16
5.0	0.00	0.00	0.00	14
6.0	0.73	0.57	0.64	67
7.0	0.56	0.71	0.63	51
8.0	0.92	0.57	0.71	21
9.0	0.83	0.36	0.50	14
10.0	0.58	0.56	0.57	39

accuracy				0.62	426		
macro avg				0.56	0.44	0.46	426
weighted avg				0.61	0.62	0.59	426

```
DecisionTree (pca, {'max_depth': 10, 'min_samples_split': 10})
precision    recall  f1-score   support
```

1.0	0.58	0.62	0.60	77
2.0	0.80	0.81	0.80	116
3.0	0.50	0.45	0.48	11
4.0	0.54	0.44	0.48	16
5.0	0.33	0.29	0.31	14
6.0	0.73	0.79	0.76	67
7.0	0.73	0.69	0.71	51
8.0	0.89	0.76	0.82	21
9.0	0.64	0.50	0.56	14
10.0	0.57	0.59	0.58	39

accuracy				0.69	426		
macro avg				0.63	0.59	0.61	426
weighted avg				0.68	0.69	0.68	426

```
In [75]: for name, (Xtr, Xte) in datasets.items():
        for params in naive_byes_hyper_params:
            clf = GaussianNB(**params)
            clf.fit(Xtr, y_train)
            y_pred = clf.predict(Xte)
            acc = accuracy_score(y_test, y_pred)
            results.append(
                ("NaiveBayes", name, str(params), acc)
            )
        print(f"\nNaiveBayes ({name}, {params})")
        print(classification_report(y_test, y_pred, zero_division=0))
```

NaiveBayes (raw, {'var_smoothing': 1e-09})				
	precision	recall	f1-score	support
1.0	0.67	0.57	0.62	77
2.0	0.73	0.41	0.52	116
3.0	0.39	0.82	0.53	11
4.0	0.26	1.00	0.41	16
5.0	0.29	0.71	0.42	14
6.0	0.70	0.45	0.55	67
7.0	0.75	0.71	0.73	51
8.0	0.69	0.86	0.77	21
9.0	0.33	0.57	0.42	14
10.0	0.47	0.44	0.45	39
accuracy			0.55	426
macro avg	0.53	0.65	0.54	426
weighted avg	0.64	0.55	0.56	426

NaiveBayes (raw, {'var_smoothing': 1e-08})				
	precision	recall	f1-score	support
1.0	0.55	0.58	0.57	77
2.0	0.54	0.22	0.32	116
3.0	0.33	0.82	0.47	11
4.0	0.25	0.88	0.38	16
5.0	0.26	0.79	0.39	14
6.0	0.62	0.34	0.44	67
7.0	0.68	0.63	0.65	51
8.0	0.67	0.86	0.75	21
9.0	0.47	0.50	0.48	14
10.0	0.52	0.59	0.55	39
accuracy			0.49	426
macro avg	0.49	0.62	0.50	426
weighted avg	0.55	0.49	0.48	426

NaiveBayes (raw, {'var_smoothing': 1e-07})				
	precision	recall	f1-score	support
1.0	0.54	0.56	0.55	77
2.0	0.43	0.19	0.26	116
3.0	0.27	0.82	0.41	11
4.0	0.28	0.81	0.41	16
5.0	0.24	0.79	0.37	14
6.0	0.45	0.31	0.37	67
7.0	0.53	0.39	0.45	51
8.0	0.69	0.86	0.77	21
9.0	0.50	0.50	0.50	14
10.0	0.53	0.62	0.57	39
accuracy			0.44	426
macro avg	0.45	0.58	0.47	426
weighted avg	0.47	0.44	0.43	426

NaiveBayes (standardized, {'var_smoothing': 1e-09})				
	precision	recall	f1-score	support

1.0	0.71	0.52	0.60	77
2.0	0.64	0.49	0.56	116
3.0	0.41	0.82	0.55	11
4.0	0.23	1.00	0.37	16
5.0	0.33	0.79	0.47	14
6.0	0.90	0.27	0.41	67
7.0	0.64	0.80	0.71	51
8.0	0.93	0.62	0.74	21
9.0	0.42	0.71	0.53	14
10.0	0.47	0.41	0.44	39
accuracy			0.54	426
macro avg	0.57	0.64	0.54	426
weighted avg	0.65	0.54	0.55	426

NaiveBayes (standardized, {'var_smoothing': 1e-08})				
	precision	recall	f1-score	support
1.0	0.70	0.52	0.60	77
2.0	0.66	0.51	0.58	116
3.0	0.41	0.82	0.55	11
4.0	0.25	1.00	0.41	16
5.0	0.32	0.79	0.46	14
6.0	0.92	0.33	0.48	67
7.0	0.66	0.82	0.73	51
8.0	0.94	0.71	0.81	21
9.0	0.39	0.64	0.49	14
10.0	0.47	0.41	0.44	39
accuracy			0.56	426
macro avg	0.57	0.66	0.55	426
weighted avg	0.66	0.56	0.57	426

NaiveBayes (standardized, {'var_smoothing': 1e-07})				
	precision	recall	f1-score	support
1.0	0.70	0.52	0.60	77
2.0	0.71	0.57	0.63	116
3.0	0.41	0.82	0.55	11
4.0	0.28	0.94	0.43	16
5.0	0.32	0.79	0.46	14
6.0	0.89	0.36	0.51	67
7.0	0.71	0.88	0.79	51
8.0	0.95	0.86	0.90	21
9.0	0.36	0.57	0.44	14
10.0	0.46	0.41	0.43	39
accuracy			0.59	426
macro avg	0.58	0.67	0.57	426
weighted avg	0.68	0.59	0.60	426

NaiveBayes (normalized, {'var_smoothing': 1e-09})				
	precision	recall	f1-score	support
1.0	0.53	0.51	0.52	77
2.0	0.64	0.34	0.44	116
3.0	0.47	0.73	0.57	11

4.0	0.21	0.88	0.34	16
5.0	0.30	0.50	0.38	14
6.0	0.60	0.37	0.46	67
7.0	0.76	0.69	0.72	51
8.0	0.60	0.86	0.71	21
9.0	0.38	0.57	0.46	14
10.0	0.47	0.54	0.50	39
accuracy			0.50	426
macro avg	0.50	0.60	0.51	426
weighted avg	0.57	0.50	0.51	426

NaiveBayes (normalized, {'var_smoothing': 1e-08})					
	precision	recall	f1-score	support	
1.0	0.43	0.53	0.47	77	
2.0	0.46	0.18	0.26	116	
3.0	0.41	0.64	0.50	11	
4.0	0.21	0.88	0.34	16	
5.0	0.30	0.57	0.39	14	
6.0	0.41	0.18	0.25	67	
7.0	0.66	0.61	0.63	51	
8.0	0.58	0.86	0.69	21	
9.0	0.50	0.57	0.53	14	
10.0	0.50	0.64	0.56	39	
accuracy				0.43	426
macro avg				0.45	426
weighted avg				0.46	426

NaiveBayes (normalized, {'var_smoothing': 1e-07})					
	precision	recall	f1-score	support	
1.0	0.43	0.55	0.48	77	
2.0	0.44	0.18	0.26	116	
3.0	0.35	0.64	0.45	11	
4.0	0.20	0.75	0.32	16	
5.0	0.26	0.50	0.34	14	
6.0	0.44	0.24	0.31	67	
7.0	0.63	0.51	0.57	51	
8.0	0.56	0.86	0.68	21	
9.0	0.50	0.50	0.50	14	
10.0	0.51	0.67	0.58	39	
accuracy				0.43	426
macro avg				0.43	426
weighted avg				0.46	426

NaiveBayes (pca, {'var_smoothing': 1e-09})					
	precision	recall	f1-score	support	
1.0	0.54	0.62	0.58	77	
2.0	0.62	0.67	0.65	116	
3.0	0.56	0.45	0.50	11	
4.0	0.69	0.56	0.62	16	
5.0	0.38	0.36	0.37	14	
6.0	0.62	0.37	0.47	67	

7.0	0.58	0.71	0.64	51
8.0	0.89	0.76	0.82	21
9.0	0.80	0.29	0.42	14
10.0	0.58	0.77	0.66	39
accuracy			0.60	426
macro avg	0.63	0.56	0.57	426
weighted avg	0.61	0.60	0.59	426

NaiveBayes (pca, {'var_smoothing': 1e-08})

	precision	recall	f1-score	support
1.0	0.54	0.62	0.58	77
2.0	0.62	0.67	0.65	116
3.0	0.56	0.45	0.50	11
4.0	0.69	0.56	0.62	16
5.0	0.38	0.36	0.37	14
6.0	0.62	0.37	0.47	67
7.0	0.58	0.71	0.64	51
8.0	0.89	0.76	0.82	21
9.0	0.80	0.29	0.42	14
10.0	0.58	0.77	0.66	39
accuracy			0.60	426
macro avg	0.63	0.56	0.57	426
weighted avg	0.61	0.60	0.59	426

NaiveBayes (pca, {'var_smoothing': 1e-07})

	precision	recall	f1-score	support
1.0	0.54	0.62	0.58	77
2.0	0.62	0.67	0.65	116
3.0	0.56	0.45	0.50	11
4.0	0.69	0.56	0.62	16
5.0	0.38	0.36	0.37	14
6.0	0.62	0.37	0.47	67
7.0	0.58	0.71	0.64	51
8.0	0.89	0.76	0.82	21
9.0	0.80	0.29	0.42	14
10.0	0.58	0.77	0.66	39
accuracy			0.60	426
macro avg	0.63	0.56	0.57	426
weighted avg	0.61	0.60	0.59	426

```
In [76]: results_df = pd.DataFrame(results, columns=["Classifier", "DataPrep", "Params",
display(results_df)
```


	Classifier	DataPrep	Params	Accuracy
2	DecisionTree	raw	{'max_depth': 10, 'min_samples_split': 10}	0.852113
5	DecisionTree	standardized	{'max_depth': 10, 'min_samples_split': 10}	0.852113
3	DecisionTree	standardized	{'max_depth': None, 'min_samples_split': 2}	0.835681
0	DecisionTree	raw	{'max_depth': None, 'min_samples_split': 2}	0.835681
1	DecisionTree	raw	{'max_depth': 5, 'min_samples_split': 5}	0.802817
4	DecisionTree	standardized	{'max_depth': 5, 'min_samples_split': 5}	0.802817
8	DecisionTree	normalized	{'max_depth': 10, 'min_samples_split': 10}	0.788732
6	DecisionTree	normalized	{'max_depth': None, 'min_samples_split': 2}	0.765258
7	DecisionTree	normalized	{'max_depth': 5, 'min_samples_split': 5}	0.758216
11	DecisionTree	pca	{'max_depth': 10, 'min_samples_split': 10}	0.685446
9	DecisionTree	pca	{'max_depth': None, 'min_samples_split': 2}	0.673709
10	DecisionTree	pca	{'max_depth': 5, 'min_samples_split': 5}	0.617371
23	NaiveBayes	pca	{'var_smoothing': 1e-07}	0.600939
21	NaiveBayes	pca	{'var_smoothing': 1e-09}	0.600939
22	NaiveBayes	pca	{'var_smoothing': 1e-08}	0.600939
17	NaiveBayes	standardized	{'var_smoothing': 1e-07}	0.591549
16	NaiveBayes	standardized	{'var_smoothing': 1e-08}	0.561033
12	NaiveBayes	raw	{'var_smoothing': 1e-09}	0.551643
15	NaiveBayes	standardized	{'var_smoothing': 1e-09}	0.542254
18	NaiveBayes	normalized	{'var_smoothing': 1e-09}	0.502347
13	NaiveBayes	raw	{'var_smoothing': 1e-08}	0.488263
14	NaiveBayes	raw	{'var_smoothing': 1e-07}	0.441315
19	NaiveBayes	normalized	{'var_smoothing': 1e-08}	0.434272
20	NaiveBayes	normalized	{'var_smoothing': 1e-07}	0.427230

```
In [77]: rf = RandomForestClassifier(n_estimators=100, random_state=seed)
rf.fit(X_train_std, y_train)
y_pred_rf = rf.predict(X_test_std)
print("\nRandom Forest (standardized)")
print(classification_report(y_test, y_pred_rf, zero_division=0))
```

Random Forest (standardized)				
	precision	recall	f1-score	support
1.0	0.88	0.87	0.88	77
2.0	0.88	0.94	0.91	116
3.0	0.83	0.91	0.87	11
4.0	1.00	0.62	0.77	16
5.0	0.80	0.57	0.67	14
6.0	0.93	0.97	0.95	67
7.0	0.89	0.94	0.91	51
8.0	1.00	0.86	0.92	21
9.0	0.93	0.93	0.93	14
10.0	0.92	0.90	0.91	39
accuracy			0.90	426
macro avg	0.91	0.85	0.87	426
weighted avg	0.90	0.90	0.90	426

```
In [78]: svm = SVC(kernel='rbf', C=1, random_state=seed)
svm.fit(X_train_std, y_train)
y_pred_svm = svm.predict(X_test_std)
print("\nSVM (standardized)")
print(classification_report(y_test, y_pred_svm, zero_division=0))
```

SVM (standardized)				
	precision	recall	f1-score	support
1.0	0.67	0.83	0.74	77
2.0	0.85	0.91	0.88	116
3.0	0.71	0.45	0.56	11
4.0	1.00	0.38	0.55	16
5.0	1.00	0.36	0.53	14
6.0	0.85	0.85	0.85	67
7.0	0.81	0.86	0.84	51
8.0	0.95	0.86	0.90	21
9.0	1.00	0.36	0.53	14
10.0	0.67	0.77	0.71	39
accuracy			0.80	426
macro avg	0.85	0.66	0.71	426
weighted avg	0.82	0.80	0.79	426

```
In [79]: # BONUS: łagodzenie przeuczenia dla drzewa (pruning)
clf_no_prune = DecisionTreeClassifier(random_state=seed, max_depth=None)
clf_no_prune.fit(X_train_std, y_train)
y_pred_no_prune = clf_no_prune.predict(X_test_std)
acc_no_prune = accuracy_score(y_test, y_pred_no_prune)

clf_prune = DecisionTreeClassifier(random_state=seed, max_depth=5)
clf_prune.fit(X_train_std, y_train)
y_pred_prune = clf_prune.predict(X_test_std)
acc_prune = accuracy_score(y_test, y_pred_prune)

print(f"\nDecisionTree bez przycinania: {acc_no_prune:.3f}")
print(f"DecisionTree z przycinaniem (max_depth=5): {acc_prune:.3f}")
```

DecisionTree bez przycinania: 0.836
DecisionTree z przycinaniem (max_depth=5): 0.803

Ocena klasyfikacji i interpretacja wyników

Na podstawie przeprowadzonej analizy i zebranych wyników (tabela `results_df` oraz raporty klasyfikacji) można wyciągnąć następujące wnioski:

1. Wpływ przygotowania danych:

- **Decision Trees:**
 - Najlepsze wyniki dla drzew decyzyjnych uzyskano na danych **surowych (raw)** oraz **standaryzowanych (standardized)**, osiągając dokładność (accuracy) na poziomie około **0.85**. Standaryzacja nie przyniosła w tym przypadku znaczącej poprawy, co sugeruje, że drzewa decyzyjne są stosunkowo odporne na różnice w skalach cech.
 - **Normalizacja (normalized)** znacząco pogorszyła wyniki drzew decyzyjnych (accuracy ok. 0.76-0.79). Może to wynikać z faktu, że normalizacja zmienia relacje między wartościami cech w sposób, który utrudnia drzewu znalezienie optymalnych punktów podziału.
 - **PCA (redukcja wymiarowości)** również obniżyła skuteczność drzew (accuracy ok. 0.67-0.69). Utrata części informacji podczas redukcji wymiarowości okazała się szkodliwa dla tego klasyfikatora.
- **Naive Bayes (GaussianNB):**
 - Naiwny Bayes generalnie osiągnął znacznie niższe wyniki niż drzewa decyzyjne, z maksymalną dokładnością w okolicach **0.60** dla danych po **PCA**.
 - Dla danych **surowych (raw)**, dokładność była niższa (ok. 0.44-0.55).
 - **Standaryzacja (standardized)** przyniosła niewielką poprawę (accuracy ok. 0.54-0.59).
 - **Normalizacja (normalized)** dała jedno z najgorszych wyników (ok. 0.43-0.50).

2. Wpływ hiperparametrów:

- **Drzewa Decyzyjne:**
 - Dla danych surowych i standaryzowanych, najlepsze wyniki (accuracy ~0.85) dała konfiguracja `{ 'max_depth': 10, 'min_samples_split': 10 }`.
 - Domyślne parametry (`'max_depth': None, 'min_samples_split': 2`) dawały nieco gorsze wyniki (accuracy ~0.84), co może sugerować lekkie przeuczenie w przypadku nieograniczonej głębokości.
 - Znaczne ograniczenie głębokości drzewa (`'max_depth': 5, 'min_samples_split': 5`) obniżyło dokładność do ok. 0.80. Optymalna złożoność drzewa wydaje się leżeć w okolicach głębokości 10 dla tego zbioru.
- **Naiwny Klasyfikator Bayesa (GaussianNB):**
 - Zmiana parametru `var_smoothing` miała niewielki wpływ na wyniki, z minimalnymi różnicami w dokładności. Dla danych po PCA, wszystkie testowane

wartości `var_smoothing` dały identyczną dokładność (0.60). Dla innych typów przygotowania danych, różnice były minimalne.

3. Porównanie klasyfikatorów:

- **Drzewa Decyzyjne** zdecydowanie przewyższyły **Naiwny Klasyfikator Bayesa** pod względem dokładności i innych metryk (F1-score, precision, recall) dla większości klas. Wynika to prawdopodobnie z faktu, że założenie NB o warunkowej niezależności cech jest rzadko spełnione w rzeczywistych zbiorach danych, a drzewa są w stanie modelować bardziej złożone zależności.

4. Radzenie sobie z klasami mniejszościowymi:

- Dodatkowo można zauważyć, że:
 - **Drzewa Decyzyjne (zwłaszcza z `max_depth=10`)** radziły sobie stosunkowo dobrze z większością klas, w tym niektórymi mniejszościowymi. Na przykład, dla klasy 3 (11 próbek w teście), drzewo z `max_depth=10` na danych surowych osiągnęło recall 0.91 i precision 0.67. Dla klasy 9 (14 próbek), recall 0.93 i precision 1.00. Jednak dla klasy 5 (14 próbek), wyniki były słabsze (recall 0.43, precision 0.55).
 - **Naiwny Klasyfikator Bayesa** miał większe problemy z klasami mniejszościowymi. Często osiągał wysoki recall kosztem bardzo niskiej precyzji (np. dla klasy 4 na danych surowych: recall 1.00, ale precision 0.26) lub odwrotnie, co skutkowało niskim F1-score. To typowe dla niezbalansowanych danych, gdzie model może "nauczyć się" przypisywać próbki do klasy mniejszościowej zbyt liberalnie lub zbyt konserwatywnie.

5. Wyniki klasyfikatorów bonusowych (Random Forest, SVM):

- **Random Forest:** Na danych standaryzowanych osiągnął **najwyższą dokładność spośród wszystkich testowanych modeli (ok. 0.90)**. Random Forest, jako zespół drzew decyzyjnych, często wykazuje lepszą generalizację i odporność na przeuczenie niż pojedyncze drzewo. Wysokie wartości precision i recall dla większości klas potwierdzają jego skuteczność.
- **SVM (Support Vector Machine):** Na danych standaryzowanych (co jest zalecane dla SVM) uzyskał dokładność na poziomie 0.80. Jest to wynik porównywalny z niektórymi konfiguracjami drzew decyzyjnych, ale niższy niż Random Forest. SVM również miał problemy z niektórymi klasami mniejszościowymi (np. niski recall dla klasy 9).

6. Łagodzenie przeuczenia (przycinanie drzewa):

- Eksperyment z przycinaniem drzewa (ograniczenie `max_depth=5`) na danych standaryzowanych pokazał, że dokładność spadła z 0.836 (dla drzewa nieprzycinanego, `max_depth=None`) do 0.803.

- W tym konkretnym przypadku, głębsze drzewo lepiej poradziło sobie na zbiorze testowym. Oznacza to, że albo nie doszło do znaczącego przeuczenia w drzewie o nieograniczonej głębokości na tym podziale danych, albo `max_depth=5` było zbyt restrykcyjnym ograniczeniem, prowadząc do niedouczenia (underfitting) i utraty zdolności do wychwycenia istotnych wzorców.

Wnioski

Podsumowanie i wnioski końcowe

Przeprowadzone ćwiczenie pozwoliło na praktyczne zapoznanie się z podstawowymi krokami projektu opartego o uczenie maszynowe, od eksploracji danych po ocenę modeli.

Kluczowe wnioski z analizy danych kardiologicznych to:

1. **Najlepszy model:** Spośród testowanych konfiguracji, **Random Forest** na danych standaryzowanych osiągnął najwyższą ogólną dokładność (ok. 90%), wykazując dobrą zdolność do klasyfikacji większości typów wzorców morfologicznych płodu.
2. **Decision Trees** również okazały się skuteczne, szczególnie z odpowiednio dobranymi hiperparametrami (np. `max_depth=10`, `min_samples_split=10`), osiągając dokładność do 85% na danych surowych lub standaryzowanych.
3. **Naive Bayes** wypadł znacznie słabiej, co prawdopodobnie wynika z niespełnienia jego założenia o niezależności cech. Niewielką poprawę przyniosła standaryzacja lub redukcja wymiarowości (PCA).
4. **Przygotowanie danych:** Standaryzacja była korzystna lub neutralna dla większości modeli. Normalizacja generalnie pogarszała wyniki. PCA dało mieszane rezultaty, pomagając NB, ale szkodząc drzewom.
5. **Niezbalansowanie klas:** Zbiór danych jest niezbalansowany, co stanowiło wyzwanie, szczególnie dla klas mniejszościowych. Lepsze modele (Random Forest, dobrze dostrojone Decision Trees) radziły sobie z tym lepiej, ale problem nadal jest widoczny w metrykach dla niektórych klas.
6. **Hiperparametry:** Dobór hiperparametrów ma kluczowe znaczenie. Dla drzew decyzyjnych, odpowiednie ograniczenie złożoności (np. przez `max_depth` i `min_samples_split`) jest ważne, aby uniknąć zarówno przeuczenia, jak i niedouczenia.